Plano de Migração

Identidade visual e marca — Projeto Hackudinhos

Documento de planejamento. **Nada foi alterado ainda.**

Objetivo: unificar a identidade visual do jogo com a do site, e padronizar o nome do produto como **Hackudinhos** em toda parte.

---

1. Diagnóstico — o que o levantamento mostrou

Três descobertas mudam bastante o tamanho estimado do trabalho.

O jogo já se chama Hackudinhos

O nome já está aplicado na interface do jogo (`app/layout.tsx`, `Sidebar.tsx`, `MobileTopBar.tsx`, dicionários de tradução) e — o mais importante — **no cookie de sessão**:

```
apps/api/src/auth/auth.constants.ts
export const SESSION_COOKIE_NAME = "hackudinhos_session";
```

Ou seja: quem precisa migrar de nome é o **site** e a **infraestrutura**, não o jogo. E o cookie, que seria o item mais delicado da migração, já está no nome final. Não encostar nele.

O jogo não tem uma única cor literal

Medição em `apps/jogo` (57 arquivos `.tsx`)	Quantidade
Usos de token semântico (`bg-surface`, `text-text-muted`…)	419
Classes de cor literal (`bg-slate-800`, `text-green-500`…)	**0**
Valores hex escritos direto no `.tsx`	86

Isso é a diferença entre um mutirão e uma edição pontual. Como todo componente consome tokens semânticos, **trocar a identidade visual é reescrever os valores dos tokens num arquivo só** — `apps/jogo/app/globals.css` — sem tocar em nenhum dos 57 componentes.

Os 86 hex se concentram em quatro arquivos:

Arquivo	Hex	Situação
`components/ui/glossary-manual.tsx`	40	Estética de pergaminho, deliberada
`components/ui/lofi-player.tsx`	14	A avaliar
`components/ui/language-selector.tsx`	6	A avaliar
`components/ui/theme-toggle.tsx`	4	A avaliar

As duas paletas partem de premissas diferentes

	Site (CIFRA)	Jogo (Hackudinhos)
Versão do Tailwind	3 (config em `.ts`)	4 (config em CSS, `@theme`)
Temas	Escuro fixo	**Claro e escuro**
Nomes dos tokens	Por cor (`chumbo`, `menta`)	Por função (`bg`, `primary`)
Cor de erro	Não existe	`--color-danger`

As duas últimas linhas são as que geram decisão de verdade, e estão na seção 5.

---

2. Parte A — Identidade visual

A.1 Mapeamento de tokens

A paleta do site, aplicada aos tokens do jogo:

Token do jogo	Valor proposto	Origem no site
`--color-bg`	`#1B1D21`	chumbo
`--color-surface`	`#2E3138`	chumbo.painel
`--color-surface-2`	`#383C44`	chumbo.linha
`--color-surface-3`	`#4A4F59`	chumbo.linhaForte
`--color-border`	`#4A4F59`	chumbo.linhaForte
`--color-primary`	`#3ED6A4`	menta
`--color-primary-hover`	`#61DDB4`	menta.400
`--color-success`	`#3ED6A4`	menta
`--color-warning`	`#F2A93D`	âmbar
`--color-danger`	*a definir*	— (ver seção 5)
`--color-text`	`#F2F4F3`	papel
`--color-text-muted`	`#A9AFAE`	papel.2

O petróleo (`#164752`) não entra como tinta: no site ele é campo institucional, usado só como fundo. Serviria como fundo de destaque em painéis do jogo.

A.2 Passos

1. **Reescrever os tokens** em `apps/jogo/app/globals.css`, no bloco do tema escuro. É a mudança central, e são cerca de 12 linhas.

2. **Resolver o tema claro** conforme a decisão da seção 5.
3. **Avaliar os quatro arquivos com hex.** O `glossary-manual` provavelmente deve ser preservado como está — a estética de pergaminho é uma peça de design intencional, não descuido. Os outros três merecem análise caso a caso.
4. **Conferir contraste.** O site foi construído com razões de contraste verificadas (papel sobre chumbo = 15,4:1; menta = 9,2:1; âmbar = 8,5:1). Os mesmos pares precisam ser reconferidos no contexto do jogo, porque lá há superfícies sobrepostas que o site não tem.
5. **Revisar tela a tela.** São 15 telas; a passagem visual é o que pega o que a medição não pega.

A.3 Por que isso não é uma reescrita

Vale registrar, porque a intuição diz o contrário: os componentes não mudam. Um `<div className="bg-surface text-text">` continua idêntico — só o valor por trás de `surface` e `text` é outro. É exatamente o retorno de ter usado tokens semânticos desde o começo.

---

3. Parte B — Renomeação para Hackudinhos

Os nomes existentes se dividem em três categorias, e a diferença entre elas é o que evita quebrar coisa em produção.

Categoria 1 — Nome de exibição (seguro)

Texto que o usuário lê. Trocar não quebra nada.

Onde	O que tem
`apps/site/src/app/layout.tsx`	Título e descrição das páginas
`apps/site/src/components/layout/Marca.tsx`	A marca no cabeçalho
`apps/site/src/components/layout/Footer.tsx`	Rodapé
`apps/site/src/app/sobre/`, `/contato/`, `/treinamento/`	Corpo de texto
`README.md`	Documentação
`Contexto Claude/contexto.md`	Documentação interna

Categoria 2 — Identificador interno (contido)

Não aparece para o usuário, mas está no código. Trocar exige rodar o build.

Onde	O que tem	Observação
`package.json` (raiz)	`"name": "cifra"`	Nome do workspace

Onde	O que tem	Observação
`Jogo/project.godot`	`config/name="CyberGuard"`	Protótipo, fora do ar
`deploy/*.service`	Nomes de serviço systemd	**Legado** — ver seção 6

Categoria 3 — Identificador externo (quebra se mudar)

Aqui mora o risco. São nomes dos quais outras coisas dependem.

Item	Valor atual	Consequência de mudar
Projeto Vercel da API	`cyber-guard-api`	Muda a URL pública
Projeto Vercel do jogo	`cyber-guard-jogo`	Muda a URL pública
Projeto Vercel do site	`cyber-guard`	Muda a URL pública
`apps/jogo/next.config.ts`	`API_PRODUCAO = "https://cyber-guard-api.vercel.app"`	**O jogo perde a API**
Cookie de sessão	`hackudinhos_session`	**Já está correto — não mexer**

O item crítico é a relação entre os dois últimos: o jogo tem a URL da API escrita no código como fallback. Renomear o projeto da API na Vercel sem atualizar esse arquivo **derruba o login em produção**.

B.1 Ordem segura para a renomeação externa

Se os projetos da Vercel forem renomeados, esta ordem evita janela de quebra:

1. Renomear o projeto da **API** na Vercel e anotar a URL nova
2. A URL antiga continua respondendo por um tempo — confirmar isso antes de seguir
3. Atualizar `API_PRODUCAO` em `apps/jogo/next.config.ts` e a variável `API_ORIGIN` no projeto do jogo
4. Publicar o jogo e rodar `node scripts/verificar-api.js <url-nova>`
5. Só então renomear os projetos do **jogo** e do **site**, que ninguém referencia por código

Alternativa mais simples: não renomear nada na Vercel. As URLs não precisam casar com o nome do produto, e é possível apontar um domínio próprio para os projetos depois — o que resolve a aparência sem mexer em identificador nenhum.

---

4. Ordem de execução sugerida

As duas partes são independentes e podem ser feitas em qualquer ordem, mas convém separá-las em commits distintos: se algo quebrar, saber se foi a cor ou o nome economiza tempo.

Etapa	Escopo	Risco
1	Tokens de cor do jogo (`globals.css`)	Baixo
2	Revisão visual das 15 telas	Baixo
3	Arquivos com hex, caso a caso	Baixo
4	Nomes de exibição no site	Baixo
5	Identificadores internos	Médio (exige build)
6	Renomeação na Vercel, se for o caso	**Alto**

Etapas 1 a 5 são seguras e locais. A 6 mexe em produção e merece ser feita separada, num momento sem demonstração marcada.

---

5. Decisões que precisam ser tomadas antes

Estas três não têm resposta técnica única — dependem do que se quer.

5.1 O jogo mantém o tema claro?

O jogo hoje tem tema claro e escuro, com um seletor (`components/ui/theme-toggle.tsx`). O site é escuro fixo, e a paleta CIFRA foi desenhada só para fundo escuro — não existe versão clara dela.

- **Manter os dois temas:** exige criar uma variante clara da identidade CIFRA, o que é trabalho de design, não de código.
- **Só escuro, como o site:** unifica de verdade e é mais rápido, mas remove um recurso que já existe e funciona.

5.2 Qual será a cor de erro?

O site não tem cor de erro. Ele usa o âmbar para risco, sempre em bloco preenchido. O jogo tem `--color-danger` (hoje `#d97a68`) e usa em situações que são genuinamente erro — resposta errada, falha de operação.

Usar âmbar para as duas coisas apagaria a diferença entre "atenção" e "errou". O provável é precisar de um vermelho novo, harmonizado com a paleta e com contraste verificado sobre o chumbo.

5.3 As URLs mudam?

Ver a alternativa na seção B.1. Renomear projeto na Vercel é a única etapa de risco alto do plano inteiro, e é opcional.

---

6. Achados laterais

- `deploy/` **parece legado.** Contém `cyberguard-api.service`, `cyberguard-web.service` e uma configuração nginx para `play.mizanis.com.br`. São artefatos de uma hospedagem em servidor próprio, anterior à Vercel. Vale confirmar com a equipe e provavelmente remover — hoje eles só geram ocorrências falsas na busca por nomes.
- **O protótipo Godot** tem `config/name="CyberGuard"`. Como está fora do ar, entra junto por consistência, sem urgência.
- **Nenhuma tabela ou coluna do banco carrega nome de marca**, o que significa que a renomeação não toca em dados nem exige migração.

---

7. Como verificar que deu certo

Verificação	Como
Build dos três apps	`pnpm run build`
Tipos	`pnpm run typecheck`
Testes das regras de jogo	`pnpm run test`
API roteando	`node scripts/verificar-api.js`
Caminho da demo	`node scripts/verificar-api.js https://<url-do-jogo>`
Sobrou nome antigo?	`grep -ri "cifra\`
Contraste	Conferir os pares texto/fundo contra WCAG AA (4,5:1)

A busca final por nomes antigos é o que fecha a migração: enquanto ela retornar algo fora de `docs/` e do histórico, ainda há ponta solta.

---

8. Resumo do esforço

Parte	Arquivos afetados	Observação
Identidade visual	1 principal + até 4 a avaliar	Concentrado no `globals.css`
Nomes de exibição	~8	Todos no site
Identificadores internos	3	Build necessário
Infraestrutura (opcional)	1 + painel Vercel	Única etapa de risco alto

O trabalho é menor do que a descrição sugere, e a razão é estrutural: o jogo foi construído com tokens semânticos, e o nome final já está aplicado nele. O que resta é o site e a decisão sobre tema claro e cor de erro.